Reviewer Pack — Schur-Weyl Decomposition Dimension Bounds Resolution Width o…

Entry 2b6e5f4e4eb7 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-20 18:50:24 UTC

Schur-Weyl Decomposition Dimension Bounds Resolution Width of 3-CNFs

Entry ID: `2b6e5f4e4eb7` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-20 18:50:24 UTC

1. Conjecture

Field A (mathematical branch): Schur-Weyl Duality **Field B** (complexity object): Resolution Width

Statement:

> For a random 3-CNF formula Φ on n variables, let d(Φ) be the dimension of the irreducible GL_n-module in the symmetric square of its incidence tensor. Then resolution width w(Φ) ≥ Ω(d(Φ)^(1/3) / log n).

Rationale (proposer's reasoning):

> Schur-Weyl duality decomposes tensor spaces into irreducible representations, revealing algebraic structure. High-dimension GL_n-modules in symmetric squares may correlate with combinatorial complexity, as resolution width measures the minimal branching program size for CNF satisfiability.

Taxonomy category: `GCT_DET_PERM` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `adf134cb6dca4207`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM_1	LLM_2
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
from fractions import Fraction
import math
import itertools

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def incidence_tensor(clauses):
        n = max(abs(v) for v in set.union(*clauses))
        tensor = [[0] * (n + 1) for _ in range(n + 1)]
        for clause in clauses:
            for i, var in enumerate(clause):
                if var > 0:
                    tensor[var][var - i] += 1
                else:
                    tensor[-var][-var + i] += 1
        return tensor

    def symmetric_square(tensor):
        n = len(tensor)
        result = [[0] * (n + 1) for _ in range(n + 1)]
        for i in range(1, n + 1):
            for j in range(i, n + 1):
                result[i][j] += tensor[i][k] * tensor[j][k] for k in range(1, n + 1)
                result[j][i] = result[i][j]
        return result

    def young_tableaux_count(n, m):
        if n == 0 and m == 0:
            return 1
        if n < 0 or m < 0:
            return 0
        return young_tableaux_count(n - 1, m) + young_tableaux_count(n, m - 1)

    def resolution_width(clauses):
        # Simplified DPLL-style width analysis for demonstration purposes
        max_width = 0
        stack = [(clauses, [])]
        while stack:
```

```
            clauses, assignment = stack.pop()
            if not clauses:
                max_width = max(max_width, len(assignment))
                continue
            unit_clause = next((c for c in clauses if len(c) == 1), None)
            if unit_clause:
                var = unit_clause[0]
                new_assignment = assignment + [var]
                stack.append(([(v for v in c if v != var and -v not in new_assignment) for c in cl
                stack.append(([(v for v in c if v != -var and -v not in new_assignment) for c in c
            else:
                literals = set()
                for clause in clauses:
                    literals.update(clause)
                literal, _ = random.choice(list(literals), 1)
                new_assignment = assignment + [literal]
                stack.append(([(v for v in c if v != literal and -v not in new_assignment) for c i
        return max_width

    def d_phi(tensor):
        n = len(tensor)
        partition = (n-1, 1)
        dim = young_tableaux_count(*partition)
        return dim

    def generate_random_3cnf(n, m):
        variables = list(range(1, n + 1))
        clauses = []
        for _ in range(m):
            clause = random.sample(variables, 3)
            random.shuffle(clause)
            clauses.append(clause)
        return clauses

    n = 40
    m = 2 * n**2  # Approximately 2^2n clauses for a dense 3-CNF
    clauses = generate_random_3cnf(n, m)

    tensor = incidence_tensor(clauses)
    symmetric_tensor = symmetric_square(tensor)
    d_phi_value = d_phi(symmetric_tensor)
    w_phi_value = resolution_width(clauses)

    if d_phi_value == 0:
        return {
            "metric_name": "resolution_width",
            "metric_value": -1,  # Indicate invalid case
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": "d(Φ) is zero"
        }

    lower_bound = 0.8 * d_phi_value**(1/3) / math.log(n)
    conjecture_holds = w_phi_value >= lower_bound
    counterexample = "" if conjecture_holds else f"w(Φ)={w_phi_value}, lower_bound={lower_bound}"
```

```
    return {
        "metric_name": "resolution_width",
        "metric_value": w_phi_value,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else list(range(2, 30))  # Default to f

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='resolution_width' first_failing_seed={first_fail
    else:
        print("RESULT: INCONCLUSIVE reason=insufficient_support")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_02d9e98f.py", line 83, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_02d9e98f.py", line 61, in run_trial
    tensor = incidence_tensor(clauses)
             ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_02d9e98f.py", line 30, in incidence_ten
    n = max(abs(v) for v in set.union(*clauses))
                            ^^^^^^^^^^^^^^^^^^^
TypeError: descriptor 'union' for 'set' objects doesn't apply to a 'list' object
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> Test crashed due to TypeError in set.union call on list objects, preventing data collection | next: Fix the incidence_tensor function to handle list inputs properly (e.g., convert to sets first)

11. Audit log (LLM calls)

Total LLM calls: 14

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	95185
2	propose	ollama_remote	qwen3:8b	0	0	138321
3	propose	ollama_remote	qwen3:8b	0	0	125399
4	propose	ollama_remote	qwen3:8b	0	0	138646
5	propose	ollama_remote	qwen3:8b	0	0	67729
6	propose	ollama_remote	qwen3:8b	0	0	59906
7	preregistration	ollama_remote	qwen3:8b	0	0	32102
8	novelty	ollama_remote	qwen3:8b	0	0	27365
9	novelty	ollama_remote	qwen3:8b	0	0	21415
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14183
11	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12737
12	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10263
13	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14251
14	judge	ollama_remote	qwen3:8b	0	0	21385

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 778889 ms total latency. Provider mix: {'ollama_remote': 14}

(full prompt+response transcripts available in `research/audit/2b6e5f4e4eb7.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/2b6e5f4e4eb7.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/2b6e5f4e4eb7.tar.gz` (if generated)